Agnostic Lightning Network Payment Integration for Litecoin — Architecture & Implementation Report (Standalone BANKON Module)

TL;DR

- **The most viable Litecoin Lightning node is `ltcsuite/lnd` (aka `lndltc`), the Litecoin fork of Lightning Labs' LND — but this is a serious finding: mainline `lightningnetwork/lnd` REMOVED all Litecoin support in v0.18.0-beta (2024), and the `ltcsuite/lnd` fork is stale (latest tag `v0.14.2-beta.rc2.1`, ~2 years old, with a `go.mod` pinning `ltcsuite/ltcd v0.22.1-beta` and Go 1.19-era dependencies — roughly 3 years behind upstream LND, currently `v0.20.0-beta`, signed Nov 11, 2025; only 14 GitHub stars).** No actively maintained, first-class "agnostic" Lightning library treats Litecoin as a peer of Bitcoin today. Litecoin-native Lightning tooling is thin and should be flagged as a project risk, not assumed.
- **Recommended standalone architecture:** run `litecoind` (Litecoin Core 0.21.x, MWEB/Taproot) + `ltcd`/`neutrino` as chain backend, `lndltc` as the node, and expose only its invoice-scoped REST API (via `invoice.macaroon`) to a thin PHP payment server, keeping the node non-custodial and admin-key-free at the application layer. Name it in Latin per cypherpunk2048 convention — e.g. `fulmen` (Latin: "lightning").
- **Reference implementation:** PHP backend (`robclark56/lightningPay-PHP`, LND REST + invoice macaroon) for invoice generation/verification, a minimal reactive UI using `bitcoinjs/bolt11` + a QR library for BOLT11 display and status polling, tested on Litecoin `regtest`/`testnet4` before mainnet. Keep it decoupled from PARSEC/x402 now, but mirror x402's 402-gated request pattern for a future bridge.

Key Findings

Litecoin Lightning support is real but under-maintained — this is the central risk

- Litecoin was historically the *first* production chain for Lightning: SegWit activated on Litecoin on **May 10, 2017 at block 1,201,536** (per CoinDesk, "Litecoin Successfully Activates SegWit": *"Earlier today, the change activated at block 1201536"*), roughly three months before Bitcoin's SegWit activation on August 24, 2017. LND originally shipped multi-chain scaffolding supporting both chains — per the LND v0.4-beta release notes: *"This is the first release that comes enabled with a flag to run on Bitcoin's mainnet, and also Litecoin's mainnet."*
- **Mainline `lightningnetwork/lnd` removed Litecoin entirely in v0.18.0-beta.** The release notes (`release-notes-0.18.0.md`) state verbatim: *"Remove Litecoin code. With this change, the Bitcoin.Active config option is now deprecated since Bitcoin is now the only*

supported chain. The chain field in the lnrpc.Chain message has also been deprecated for the same reason." The current `lnd.conf` confirms: *"[Bitcoin] DEPRECATED: If the Bitcoin chain should be active. This field is now ignored since only the Bitcoin chain is supported."* So you **cannot** use current upstream LND for Litecoin. Lightning Engineering

- Litecoin Lightning now lives only in the community fork **`ltcsuite/lnd`** (**`lndltc`**), which is BOLT-compliant (BOLT 1–11) but stale (~3 years behind upstream, Go 1.19-era, 14 stars, 5 forks). GitHub
- **The live Litecoin Lightning network is essentially dormant.** Per 1ml.com/litecoin/statistics (observed July 6, 2026): **95 active nodes (69 public), 170 active channels, and 35.27 LTC total capacity** (*"0.000042% — 35.27 out of 84,000,000 LTC maximum supply"*), with 0.00% change over the trailing 30 days and top nodes last updated years ago. 1ML cautions that *"the numbers observed are approximations and nodes that don't broadcast their state are not included."* This means routing liquidity for *outbound* Litecoin Lightning payments to arbitrary counterparties is scarce; a merchant-receive use case (customers pay you) with your own well-funded channels is far more realistic than routing across the public LTC graph.

Implementation options compared

Option	Repo	Lang	License	Litecoin support	Maintenanc
lndltc	github.com/ltcsuite/lnd	Go	MIT	Native (purpose-built fork)	Stale (~2 yrs; v0.14.2-beta.
ltcd	github.com/ltcsuite/ltcd	Go	ISC	Native full node backend	Beta, more ac than lndltc
litecoind (Litecoin Core)	github.com/litecoin-project/litecoin	C++	MIT	Native L1 (Taproot+MWEB, 0.21.2+)	Active
neutrino (LTC)	github.com/ltcsuite/neutrino	Go	MIT	Native light client	Stale
Mainline LND	github.com/lightningnetwork/lnd	Go	MIT	**REMOVED v0.18.0**	Very active (v0.20.0-beta
Core Lightning (CLN)	github.com/ElementsProject/lightning	C	MIT/BSD-MIT	Bitcoin-only (bitcoind ≥25)	Very active

Option	Repo	Lang	License	Litecoin support	Maintenanc
LDK / rust-lightning	github.com/lightningdevkit/rust-lightning	Rust	MIT/Apache-2.0	Bitcoin-only; chain-abstracted but no LTC port	Very active
Litecoin Dev Kit (LDK-LTC)	github.com/LitecoinDevKit	Rust	Apache-2.0/MIT	**On-chain wallet only** (BDK port), NOT Lightning	Early/experir
Eclair	github.com/ACINQ/eclair	Scala	Apache-2.0	Historically LTC, now Bitcoin-focused	Active

Note on "agnostic": The truly chain-agnostic, actively maintained library is **LDK (`lightningdevkit/rust-lightning`, Apache-2.0/MIT)** — it abstracts chain access, key management, and storage — but it has **no Litecoin implementation**; the separate "Litecoin Dev Kit" (`LitecoinDevKit`) is a *BDK on-chain wallet port*, not Lightning, and is experimental. Therefore there is no drop-in agnostic Lightning library that supports Litecoin today. The pragmatic "agnostic" path is to wrap `lndltc`'s gRPC/REST API behind a chain-neutral module interface, so the same PHP/Python service could later target a Bitcoin LND or CLN node by swapping the backend connection string. Lightningdevkit GitHub

Client libraries and UI components

- **PHP payment servers (his stated preference):**
  - `robclark56/lightningPay-PHP` (github.com/robclark56/lightningPay-PHP) — eCommerce payment via LND REST + `invoice.macaroon`; the macaroon is invoice-scoped so a leak *cannot spend funds*. Based on `lightningtip-PHP`.
  - `ndeet/php-ln-lnd-rest` (github.com/ndeet/php-ln-lnd-rest) — Swagger-generated PHP client for LND's full REST API (invoices, channels, etc.).
  - `jorijn/bitcoin-bolt11` (github.com/jorijn/bitcoin-bolt11) — mature PHP BOLT11 decoder.
  - `dcentrica/lightning-bolt11` (github.com/dcentrica/lightning-bolt11) — standalone PHP BOLT11 encoder/decoder.
- **JS/UI (minimal reactive UI):**
  - `bitcoinjs/bolt11` (github.com/bitcoinjs/bolt11; npm `bolt11`) — encode/decode BOLT11, browser-buildable.
  - `andrerfneves/lightning-decoder` (github.com/andrerfneves/lightning-decoder) — React/TypeScript BOLT11/LNURL/BOLT12 decoder + QR scanner components

(shadcn/ui based).
  - A QR library (e.g. `qrcode` / `qrcode.react`) to render the `lightning:<bolt11>` URI.
- **Testing:** BTCPay's local dev docs document a Litecoin regtest Lightning flow (`docker-litecoin-cli`, Polar). `lndltc` supports `regtest`/`testnet4`. Litecoin's faster 2.5-min blocks make regtest/testnet iteration faster than Bitcoin.

Details

Recommended stack (standalone `fulmen` module)

```
[ Litecoin Core 0.21.2+ (litecoind, Taproot+MWEB) ]  <- L1 backend
                |
      [ lndltc (ltcsuite/lnd) ]  <- Lightning node, BOLT11, channels
                |  gRPC:10009 / REST:8080 (invoice.macaroon only)
      [ fulmen_server (PHP >=8, or Python >=3.12) ]  <- invoice gen/verify
                |  JSON over HTTPS
      [ fulmen_ui (minimal reactive: QR + BOLT11 + poll) ]
```

- **Node setup:** `litecoind` with `txindex=1`, `server=1`, ZMQ raw block/tx endpoints (`zmqpubrawblock=tcp://127.0.0.1:28332`, `zmqpubrawtx=tcp://127.0.0.1:28333`). `lndltc` config: `litecoin.active=1`, `litecoin.mainnet=1`, `litecoin.node=litecoind`. Data dir `~/.lndltc`. Run under **OpenBSD vmm** or a **Podman** rootless container (per his conventions), not Docker. Medium GitHub
- **Wallet/channel management:** `lncli --chain=litecoin` for wallet create/unlock, `newaddress p2wkh` for bech32 (`ltc1...`) funding, `openchannel` to a known-good, well-funded peer. Given the tiny public graph (95 nodes / 35.27 LTC), plan to open direct channels to the specific counterparties you intend to transact with, or run both ends (merchant + LSP) yourself. Medium
- **Invoice generation:** `addinvoice` (gRPC/REST) returns a BOLT11 `payment_request` (LTC HRP prefix `lnltc` / testnet `lntltc`; regtest `lnltcrt`) and an `r_hash` (payment hash) — the payment hash is your idempotency key.
- **Payment verification:** subscribe to `invoices.SubscribeSingleInvoice` (streaming) or poll `lookupinvoice` by `r_hash`; credit exactly once on `SETTLED` state. Never credit on anything but a confirmed settle event.
- **Security model:** expose only `invoice.macaroon` (read/write invoices) to the PHP layer — never `admin.macaroon`. Contracts/components carry Apache-2.0 + BANKON copyright; no admin keys and no upgradeable proxies apply to the on-chain governance contracts (BANKON stack), while the Lightning node's own key material (seed, `wallet.db`,

macaroons) is kept off the web tier and backed up via static channel backups (SCBs). Mainnet-only deployment after regtest/testnet validation. BTCPay Server

Illustrative PHP invoice-create + verify (LND REST, invoice.macaroon):

```php
<?php
// fulmen_server.php — Litecoin Lightning invoice service (invoice.macaroon only)
$LND_REST = 'https://127.0.0.1:8080';
$MACAROON = getenv('FULMEN_INVOICE_MACAROON_HEX'); // xxd -ps -u -c 1000 invoice.maca
$CERT     = '/etc/fulmen/tls.cert';

function lnd_call(string $method, string $path, ?array $body = null): array {
    global $LND_REST, $MACAROON, $CERT;
    $ch = curl_init("$LND_REST$path");
    curl_setopt_array($ch, [
        CURLOPT_CUSTOMREQUEST  => $method,
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_CAINFO         => $CERT,
        CURLOPT_HTTPHEADER     => ["Grpc-Metadata-macaroon: $MACAROON"],
        CURLOPT_POSTFIELDS     => $body ? json_encode($body) : null,
    ]);
    return json_decode(curl_exec($ch), true) ?? [];
}

// Create an invoice for N litoshis
function create_invoice(int $litoshis, string $memo): array {
    // value is in litoshis for a Litecoin lndltc node
    return lnd_call('POST', '/v1/invoices', ['value' => $litoshis, 'memo' => $memo]);
    // returns: payment_request (BOLT11, lnltc...), r_hash (base64 payment hash)
}

// Verify by payment hash (poll). Credit ONCE on state === 'SETTLED'.
function invoice_settled(string $r_hash_hex): bool {
    $r = lnd_call('GET', "/v1/invoice/$r_hash_hex");
    return ($r['state'] ?? '') === 'SETTLED';
}
```

Minimal reactive UI flow (BOLT11 + QR + status poll):

```js
```

```
// fulmen_ui.js — display BOLT11, render QR, poll settle
import { decode } from 'bolt11';          // github.com/bitcoinjs/bolt11
import QRCode from 'qrcode';              // render lightning: URI

async function showInvoice(paymentRequest, rHashHex) {
  const details = decode(paymentRequest); // validate + read amount/expiry
  document.getElementById('amt').textContent =
    (details.satoshis ?? 0) + ' litoshis';
  await QRCode.toCanvas(document.getElementById('qr'),
    'lightning:' + paymentRequest);       // BIP21-style LN URI
  const t = setInterval(async () => {
    const r = await fetch(`/api/status?h=${rHashHex}`).then(x => x.json());
    if (r.settled) { clearInterval(t); document.getElementById('st').textContent = 'P
  }, 2500);                               // 2-3s polling
}
```

Chain-registry interoperability (allchain.html / AgenticPlace)

- Litecoin is **not** an EVM chain and will not appear in `agenticplace.pythai.net/allchain.html`'s EVM-chain list. For future multi-chain awareness, register Litecoin Lightning as a distinct non-EVM rail entry using a CAIP-2-style identifier (analogous to how x402-avm uses `algorand:*` CAIP-2 network IDs). Recommended registry key: a namespaced ID like `litecoin:lightning` with SLIP-0044 coin type `2` for LTC, kept in a separate registry table from the EVM `chainId` map, so the `ChainRegistry` component in the BANKON stack can reference it without polluting the EVM chain list.

Relationship to PARSEC / x402 (reference only, NOT wired in now)

- PARSEC is his x402 rail on Algorand, built on **GoPlausible's x402-avm** (`@x402-avm/core`, `@x402-avm/avm`; Python `x402-avm` — note the import name is `x402`, not `x402_avm`). x402 gates resources with an **HTTP 402 Payment Required** response, then verifies/settles on-chain. Goplausible
- The Lightning module can *mirror* this pattern for a future bridge: return `402` with a BOLT11 invoice, then release the resource when the invoice settles. This is the **L402** pattern (formerly LSAT), developed by Lightning Labs and formalized as **bLIP-0026**. Per the Lightning Labs spec, the server *"responds with HTTP 402 Payment Required and a WWW-Authenticate header containing a token and a Lightning invoice. The token commits to the Lightning invoice by containing the invoice's payment hash."* Lightning Labs' March 11, 2026 post ("The Future Is Now") positions L402 explicitly for AI agents. **Do not integrate now** —

keep `fulmen` standalone, but design its gating interface (a `require_payment()` middleware returning 402) to be structurally compatible with a later x402↔L402 adapter.

Recommendations

Stage 1 — Prototype on regtest (no funds at risk). Stand up `litecoind -regtest` + `lndltc` in rootless Podman. Build `fulmen_server` in PHP using `robclark56/lightningPay-PHP` as the reference (invoice.macaroon only). Verify: invoice creation, QR render, settle detection via `lookupinvoice`. Use `jorijn/bitcoin-bolt11` to decode/validate invoices server-side. *Threshold to advance:* end-to-end pay + settle detection working on regtest with idempotent crediting.

Stage 2 — testnet4 validation. Point at Litecoin testnet4, open a channel to a live testnet peer or your own second node, confirm real routed settlement and SCB recovery. Build the `fulmen_ui` reactive flow (BOLT11 display, `lightning:` QR, 2–3s status polling on payment hash). *Threshold:* successful routed payment + verified recovery-from-backup drill.

Stage 3 — Mainnet, minimal capacity. Deploy on OpenBSD vmm / Podman, mainnet-only. Fund with minimal LTC, open direct channels to your intended counterparties (do not rely on the ~35 LTC public graph for routing). Expose only invoice.macaroon; store node seed and SCBs offline. *Threshold to hold/rollback:* if public LTC routing liquidity remains <50 LTC or your target counterparties aren't reachable, keep to direct/self-run channels only.

Decision triggers that change the recommendation:

- If Litecoin-native Lightning routing liquidity does not recover, **treat Litecoin Lightning as a closed-loop / direct-channel system**, not a routed network.
- If you need active upstream maintenance and security patches, **reconsider whether Bitcoin Lightning (mainline LND or CLN) better fits**, and use Litecoin only on L1 — because the `lndltc` fork carries unpatched-drift risk (~3 years behind upstream LND security fixes; upstream is at v0.20.0-beta while lndltc is pinned near v0.14).
- If a maintained agnostic library appears (e.g. an LDK Litecoin port), migrate the node layer while keeping the PHP/UI layer unchanged (that's the point of the wrapper).

Caveats

- **`lndltc` is stale and beta.** Latest tag `v0.14.2-beta.rc2.1` (~2 years old), `go.mod` pinning `ltcsuite/ltcd v0.22.1-beta` and Go 1.19-era deps — roughly 3 years behind upstream LND's security and protocol fixes. Running it on mainnet with real funds carries elevated risk; the upstream LND safety guidance (*"lnd is still beta software... ignoring these operational guidelines can lead to loss of funds"*) applies doubly.
- **Public Litecoin Lightning is near-dormant** (95 nodes / 170 channels / 35.27 LTC per 1ml.com, July 6, 2026), so this is realistically a merchant-receive or direct-channel system, not a broadly routable network. 1ml itself notes its figures are approximations. 1ML 1ml

- **No true agnostic Lightning library supports Litecoin.** LDK is chain-abstracted and actively maintained but Bitcoin-only in practice; "Litecoin Dev Kit" is an on-chain BDK port, not Lightning.
- **Compatibility gotchas:** upstream LND ⩾0.18 cannot talk to Litecoin at all; older mainline LND (⩽0.14/0.15) had a documented `unsupported net` bug (lnd issue #5909) requiring the LTC-specific fork. Bitcoin and Litecoin could never be active simultaneously in one LND instance (lnd issue #1331).
- Some third-party "2026" figures (network capacity, node counts) come from commercial/blog sources; treat non-primary numbers as indicative. Live L1 Litecoin (Taproot + MWEB) is well-maintained and current (Litecoin Core 0.21.2.x); the maintenance gap is specifically at the *Lightning* layer.